

jev-decision-service 0.1.3 · 版本对比测评报告

对比基线：0.1.0 (9/21 实测) → 0.1.1 → 0.1.3 | 测评方式：逐字节 diff + 单元测试 + 综合探针 + 服务端到端 | 2026-09-23

0 结论摘要

0.1.3 相对我 9/21 实测过的 0.1.0 有实质性但非全面的进步。最关键的一点是：这个版本是照着我们的《哨兵 EdgeSentinel 决策层评估报告》迭代出来的——README 里有一整节 “Iteration from the EdgeSentinel evaluation”，逐条复述了我们报告里的四个局限，并把它们做成了功能。

+26.8% 代码行数 (1495→1895)	+45% 单元测试 (20→29)	7 / 7 新增能力实测通过	1 遗留缺陷 (开源模型路径)
---	---	--------------------------------------	---

一句话回答「进步了多少」

维度	0.1.0 → 0.1.3	评价
代码规模	1495 → 1895 行 (+26.8%)，20 → 22 个模块	实质增长
测试覆盖	20 → 29 个用例，全绿	同步跟进
我们指出的 6 项局限	1 项真解决 · 3 项部分缓解 · 2 项未动	方向对，未走完
实现缺陷 (开源模型路径)	未修复，解析器逐字节未变	仍然不可用
架构能力	新增分层路由 / OOD 弃权 / 校准元数据	明显上台阶

1 版本谱系与规模变化

模块	0.1.0	0.1.1	0.1.3	说明
provider.py	363	447	447	新增 StructuredHeadProvider / TieredProvider
policy.py	83	95	111	OOD / 稀有类 / 未校准保护
models.py	185	192	196	新增 abstain 动作
questions.py	51	72	72	问题集可注入 (QuestionProfile)
engine.py	137	142	144	接入 warnings 与校准报告
composer.py	22	22	24	—
calibration.py	—	89	89	★ 新增：校准与先验校正
state_builder.py	—	60	77	★ 新增：状态构建与 OOD 检测
test_policy.py	46	58	76	—
test_provider.py	58	82	82	—
test_state_builder.py	—	21	27	★ 新增
合计 (Python)	1495	1830	1895	+400 行 / +26.8%

注意节奏：0.1.1 是主升级 (+335 行，新增 calibration 与 state_builder 两个模块)，0.1.3 只是微调 (+65 行，纯差分 53 行)。如果对方说「刚出了新版」，实际增量主要是 0.1.1 那批。

2 核心发现：它是照着我们的评估报告改的

README 第 62 行起有一节 “Iteration from the EdgeSentinel evaluation”，原文写道：报告显示出结构化决策头排序能力强、延迟极低，但也暴露了 class-support limits、calibration drift under a changed fault rate、rule-derived Score labels、missing unstructured-input coverage。下面是我们报告里的每一条问题，与它在代码中的对应实现。

我们报告里指出的问题	0.1.3 的对应实现	落地程度
校准随部署分布漂移 (ECE 0.0066 → 0.1675)	calibration.py 的 apply_profile：先验校正公式与我们的完全一致；DecisionHeadProfile 记录 calibration_distribution，分布不一致即报 distribution_shift	★ 真正解决
Score 标签是规则派生的，数字无含金量	label_provenance 字段 (derived_rule / synthetic_only / unknown) → 产出警告	部分：诚实标注，未解决数据源头
少数类样本不足 (TWF 6 条、RNF 6 条)	class_support + rare_class_threshold → rare_classes 警告 → 强制人工复核	部分：检测到了，数据问题依旧
未校准的概率不能用于自动决策	PolicyConfig.require_calibration_for_auto → uncalibrated_probability → 降级为 ask_clarification	★ 真正解决
阈值—自动化率权衡表 / 0.90 工作点	task_probability_thresholds + ood_review_threshold；README 明确写 0.90 是「可配置策略，而非普适准确率声明」	★ 真正解决
需要 fast head + 生成式升级的分层路由	TieredProvider (快速头低置信 → 升级到生成式) + StructuredHeadProvider (专门接收「吃数值特征、返回回答」的决策头)	★ 真正解决
非结构化输入覆盖缺失、编码器是手工特征	StateBuilder 识别 text / structured_features / invalid 三种模态，输出 routing_hint	部分
仅单一数据源 AI4I，未验证跨设备泛化	未见对应实现	未动
尚未服务化 / 无界面	未见对应实现 (仍只有 /v1/decide 等 API)	未动

值得注意的两件事：① TieredProvider + StructuredHeadProvider 简直就是为我们准备的接口——README 原话是「For a structured head supplied by later data, construct StructuredHeadProvider with an evaluator that accepts a numeric feature dictionary」。② 它把我们主动披露的三个「诚实条款」（先验漂移、派生标签、少数类）全部变成了运行时保护机制，这个思路是对的。

3 实测：六项能力逐条验证

#	验证项	方法	结果
1	单元测试	unittest discover	29 / 29 通过 (0.1.0 为 20)
2	StateBuilder · OOD 检测	注入 ood_score=0.85 / 0.30	0.85 触发 out_of_distribution，0.30 不触发 —— 正确
3	StateBuilder · 特征校验	合法数值字典 vs 含字符串的字典	分别判为 valid (routing=fast) / invalid (非数值) —— 正确
4	校准 · 先验校正数值	0.3232 + π_{train} 0.0312 + π_{deploy} 0.5，与服务输出逐位比对	服务 0.936822 $\equiv$ 手算 0.936822 —— 公式实现无误
5	校准 · 温度缩放	0.9 经 T=2.0 缩放	服务 0.750000 $\equiv$ 手算 0.750000 —— 正确
6	Policy · 保护机制	分别注入 OOD / 稀有类 / 漂移 / 未校准 / abstain	全部正确：前三者 human_review + blocked，未校准降级 ask_clarification，abstain 输出 abstain
7	TieredProvider 分层升级	快速头置信度 0.95 vs 0.40	0.95 → route=fast；0.40 → route=generative —— 正确
8	StructuredHeadProvider 接入我们自己的决策头	注入自定义 evaluator (吃数值特征)	成功产出 6 条预测，校准状态 temperature_scaled，稀有类警告正确出现在报告里

3.1 服务端到端 (rules 模式)

场景	返回动作	是否拦截	原因码
普通消息	retrieve_evidence	否	side_effect_not_authorized, history_evidence_missing
带 ood_score=0.9	human_review	是	out_of_distribution_requires_review
带 distribution_id (触发漂移)	human_review	是	distribution_shift_requires_recibration
带结构化特征	ask_clarification	否	side_effect_not_authorized, clarification_needed

新增的 OOD 与分布漂移保护在 API 层确实生效，不是只写在 README 里。/v1/replay 批量回归接口也可用（返回 count + actions 汇总）。

4 遗留缺陷：开源模型路径仍然不可用

这是我在 9/21 就实测出并修复过的问题，0.1.3 一个字都没改。

项	证据
解析器未改动	逐字节比对：_parse_jev_answer 在 0.1.0 和 0.1.3 中完全相同（2574 字节）
问题根因	解析器只接受原生键名（choice / score / noul），但 OpenSourceProvider 的提示词并未要求模型输出这些键名
实测复现（mock 真实模型输出格式）	模型返回 {"type":"choice","value":"information_request",...} → ProviderError: choice is missing from probabilities for intent
对照	模型返回原生键名 {"type":"choice","choice":"information_request"} → 解析成功，得到 6 条预测
影响	JEV_PROVIDER=opensource（对接 vLLM / Ollama 本地模型）这条路径开箱即用会直接报错；只有 rules 与官方 Jev API 两条路是完整的

这说明 0.1.x 这一系没有在真机上跑通过 opensource 路径——单元测试里对 provider 的覆盖只用了构造好的原生格式响应，而真实模型（我们实测过 27B 与 8B）不会主动输出 choice/score/noul 键名。我在 9/21 的修复方式是重写提示词、明确要求原生键名并给出示例，那三行改动至今没有被采纳。

5 对我们的意义与建议

判断	说明
① 我们的技术输出被采纳了	先验校正公式、OOD 弃权、任务工作点、分层路由——这几项正是我们方案的核心立论，现在出现在一个第三方包里，且 README 直接致谢我们的评估报告
② 但它仍停在「骨架」层	它解决的是「框架和防护」，没解决「大脑」。StructuredHeadProvider 需要一个真正的决策头喂进去——那正是我们已经训好并上线的东西
③ 可以直接对接，但要谈授权	我们的 sentinel_server.py 决策头可以通过 StructuredHeadProvider 接进它的 policy / calibration 层；但该包无 LICENSE 文件，默认保留所有权利，并入参赛作品前必须先取得授权
④ 比赛层面要留个心	这个包在跟着我们的报告快速迭代（0.1.0 → 0.1.1 → 0.1.3，两天三版）。如果它来自同赛道的其他队伍，我们的差异化窗口正在收窄——真正的护城河是我们已经跑出来的实测数据和上线运行的服务，不是架构图

建议动作：① 把这份测评作为「我们方案被第三方引用」的佐证写进作品说明，这本身就是技术影响力的证据；② 优先补上别人还追不上的部分——StepFun 接入、Nsight 剖析、演示视频；③ 若要用它的代码，先落实授权。